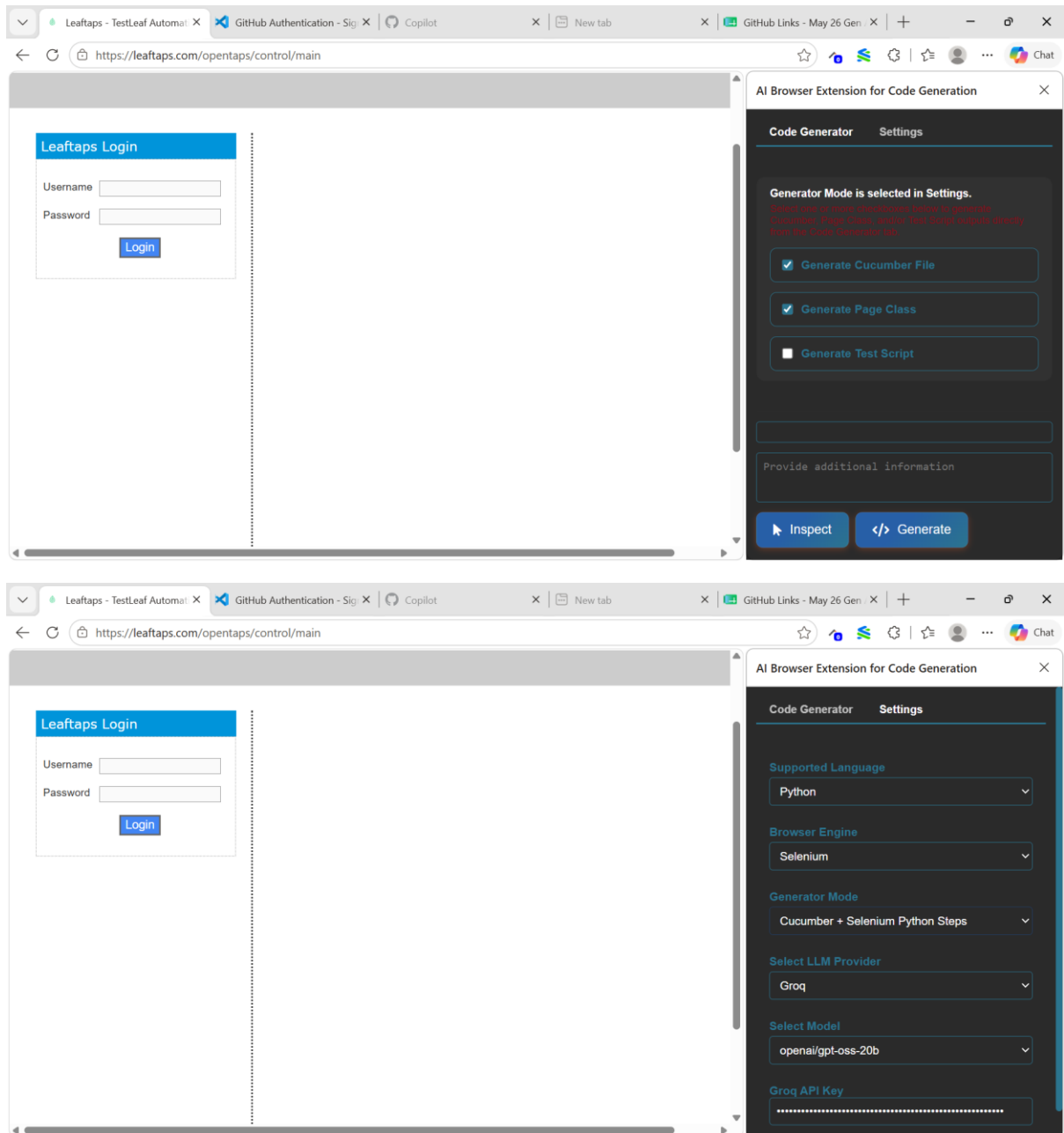
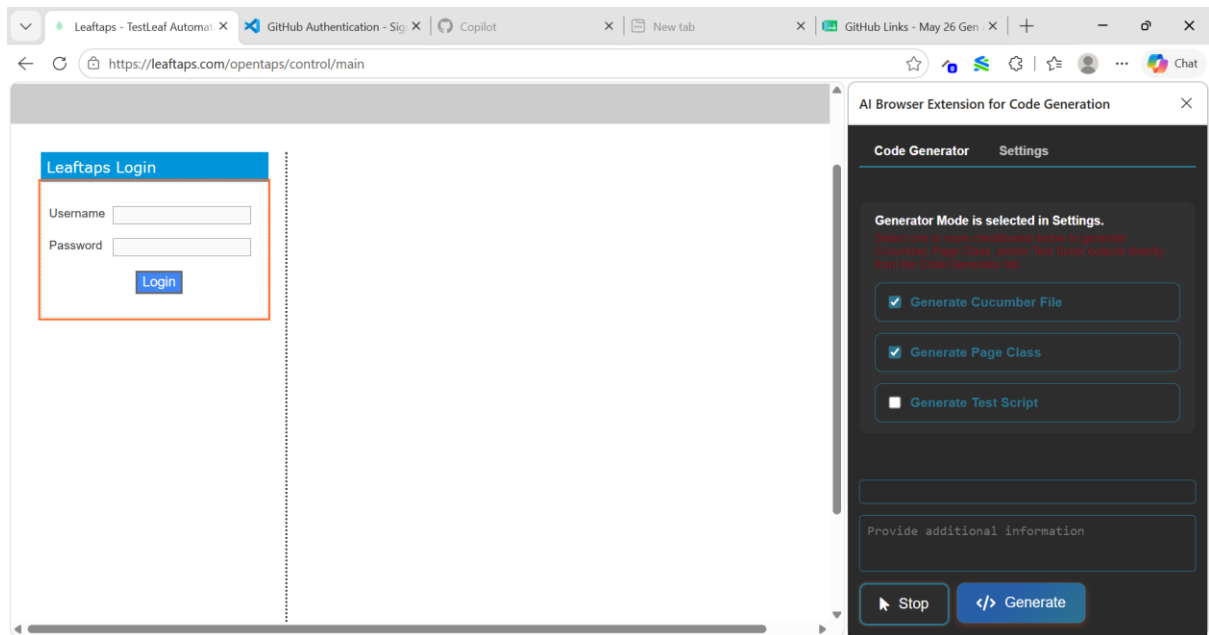


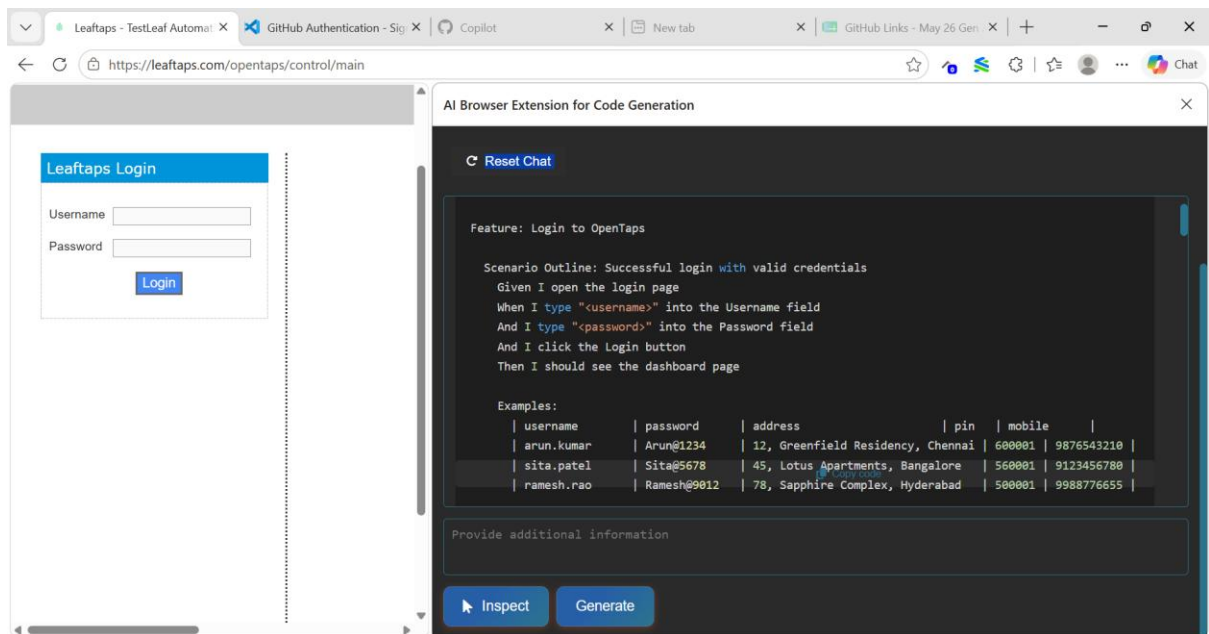
With vibe coding implement the playwright/typescript or Selenium/Python

Selenium/Python:





Feature File:



Page Class:

The screenshot shows a web browser with the URL <https://leafTaps.com/opentaps/control/main>. The main content area displays the "LeafTaps Login" form, which includes fields for "Username" and "Password", and a "Login" button. An "AI Browser Extension for Code Generation" sidebar is open on the right. The sidebar has a "Reset Chat" button and a code editor showing the following Python code:

```
from selenium.webdriver.common.by import By
from selenium.webdriver.remote.webdriver import WebDriver
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class LoginPage:
    """
    Page Object representing the login form.

    Attributes
    -----
    driver : WebDriver
        Selenium WebDriver instance used to interact with the page.
    wait : WebDriverWait
        Explicit wait helper for synchronizing element interactions.
    """
```

Below the code editor is a text input field labeled "Provide additional information" and two buttons: "Inspect" and "Generate".

This screenshot shows the same web browser and "LeafTaps Login" form as the previous one. The "AI Browser Extension for Code Generation" sidebar is still open, but the code in the editor has been updated to include locators and an initialization method:

```
# Locators
_FORM = (By.ID, "login")
_USERNAME_INPUT = (By.ID, "username")
_PASSWORD_INPUT = (By.ID, "password")
_SUBMIT_BUTTON = (By.CSS_SELECTOR, "input.decorativeSubmit")

def __init__(self, driver: WebDriver, timeout: int = 10):
    """
    Initialize the LoginPage with a WebDriver instance.

    Parameters
    -----
    driver : WebDriver
        Selenium WebDriver instance.
    timeout : int, optional
        Maximum seconds to wait for elements, by default 10
    """
```

The sidebar also features the "Reset Chat" button, the "Provide additional information" text input, and the "Inspect" and "Generate" buttons.

Leaf taps - TestLeaf Automai X GitHub Authentication - Sig X Copilot X New tab X GitHub Links - May 26 Gen X + - Chat

https://leaf taps.com/opentaps/control/main

Leaf taps Login

Username

Password

Login

AI Browser Extension for Code Generation

Reset Chat

```
Maximum seconds to wait for elements, by default 10.

self.driver = driver
self.wait = WebDriverWait(driver, timeout)

def wait_for_page_loaded(self) -> None:
    """
    Wait until the login form is visible on the page.
    """
    self.wait.until(EC.visibility_of_element_located(self._FORM))

def enter_username(self, username: str) -> None:
    """
    Enter the provided username into the username field.
    """
```

Provide additional information

Inspect Generate

Leaf taps - TestLeaf Automai X GitHub Authentication - Sig X Copilot X New tab X GitHub Links - May 26 Gen X + - Chat

https://leaf taps.com/opentaps/control/main

Leaf taps Login

Username

Password

Login

AI Browser Extension for Code Generation

Reset Chat

```
Parameters
-----
username : str
    The username to input.

username_field = self.wait.until(EC.element_to_be_clickable(self._USERNAME_INPUT))
username_field.clear()
username_field.send_keys(username)

def enter_password(self, password: str) -> None:
    """
    Enter the provided password into the password field.
    """

Parameters
-----
password : str
```

Provide additional information

Inspect Generate

LeafTaps - TestLeaf Automai X GitHub Authentication - Sig X Copilot X New tab X GitHub Links - May 26 Gen X + - Chat

https://leafTaps.com/opentaps/control/main

LeafTaps Login

Username

Password

Login

AI Browser Extension for Code Generation

Reset Chat

```
password = str
    """
    The password to input.
    """
    password_field = self.wait.until(EC.element_to_be_clickable(self._PASSWORD_INPUT))
    password_field.clear()
    password_field.send_keys(password)

def click_login(self) -> None:
    """
    Click the login button to submit the form.
    """
    submit_button = self.wait.until(EC.element_to_be_clickable(self._SUBMIT_BUTTON))
    submit_button.click()

def login_as(self, username: str, password: str) -> None:
    """
```

Provide additional information

Inspect Generate

LeafTaps - TestLeaf Automai X GitHub Authentication - Sig X Copilot X New tab X GitHub Links - May 26 Gen X + - Chat

https://leafTaps.com/opentaps/control/main

LeafTaps Login

Username

Password

Login

AI Browser Extension for Code Generation

Reset Chat

```
self.enter_username(username)
self.enter_password(password)
self.click_login()

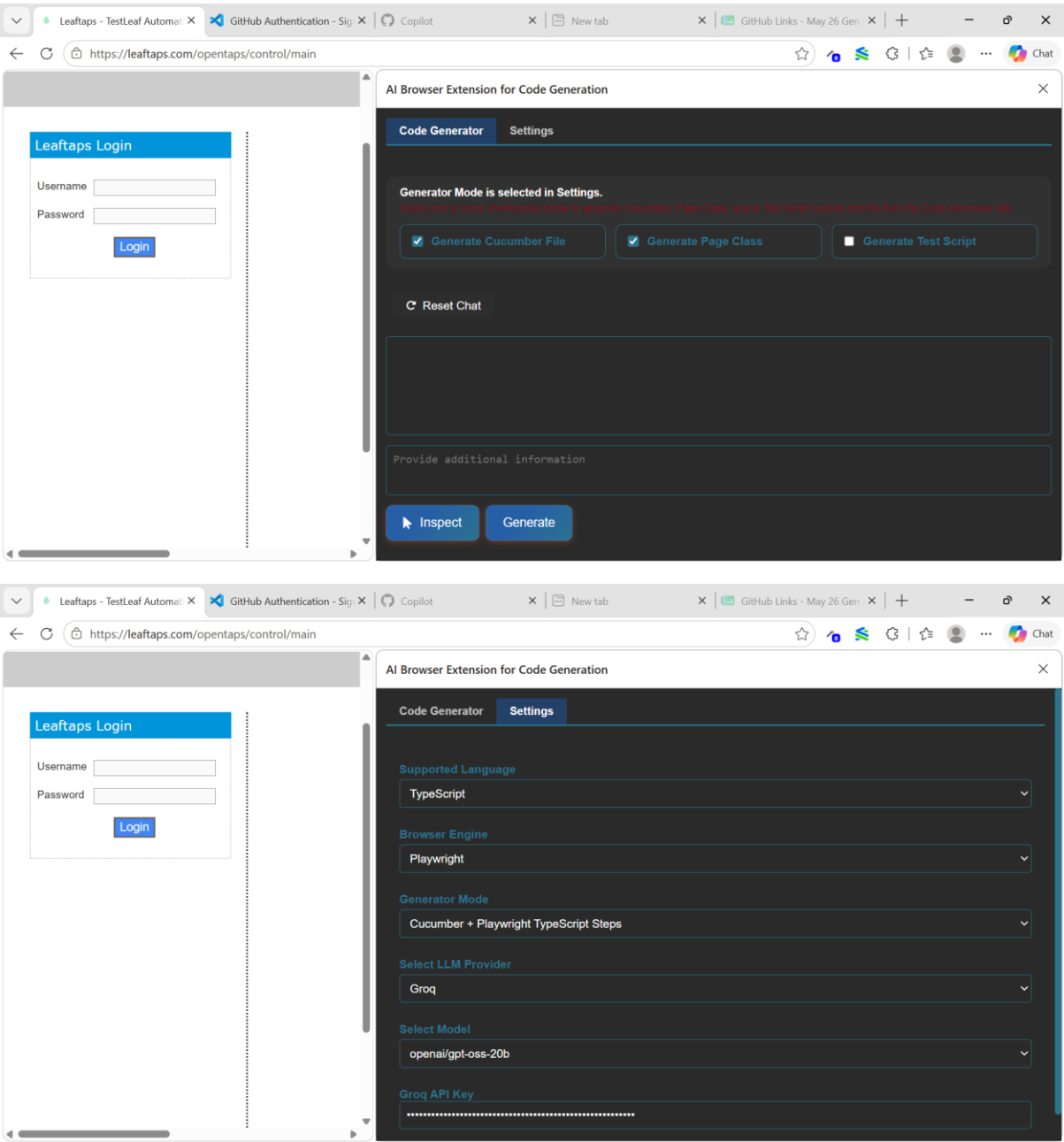
def is_login_button_enabled(self) -> bool:
    """
    Check whether the login button is enabled.

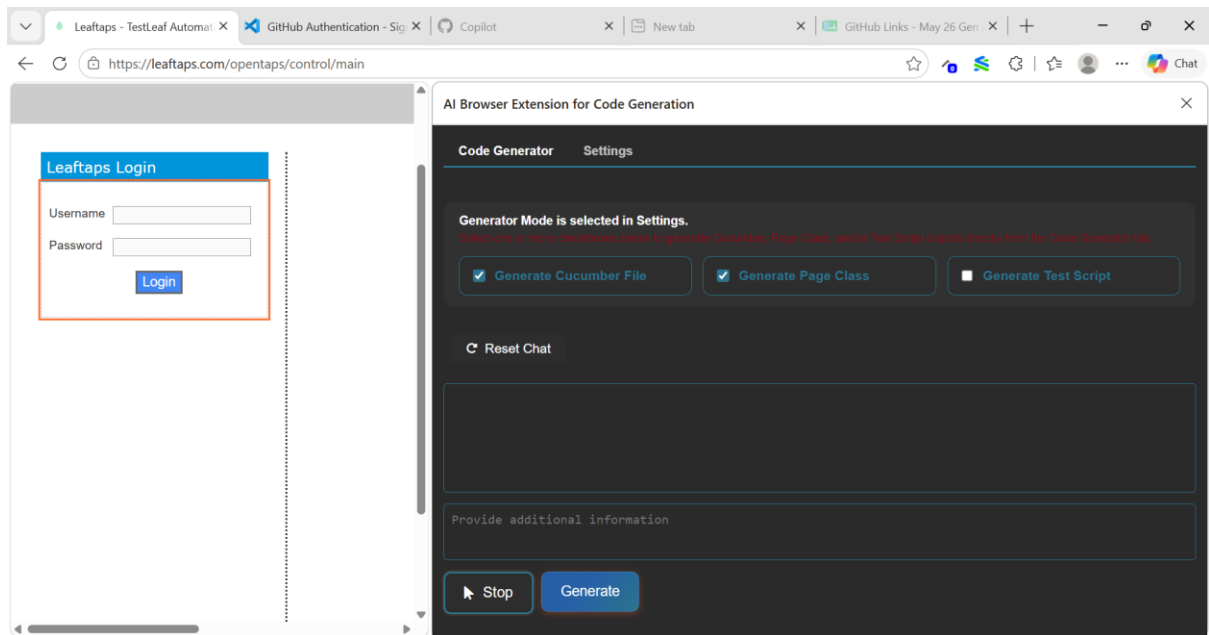
    Returns
    -----
    bool
        True if the button is enabled, False otherwise.
    """
    button = self.wait.until(EC.presence_of_element_located(self._SUBMIT_BUTTON))
    return button.is_enabled()
```

Provide additional information

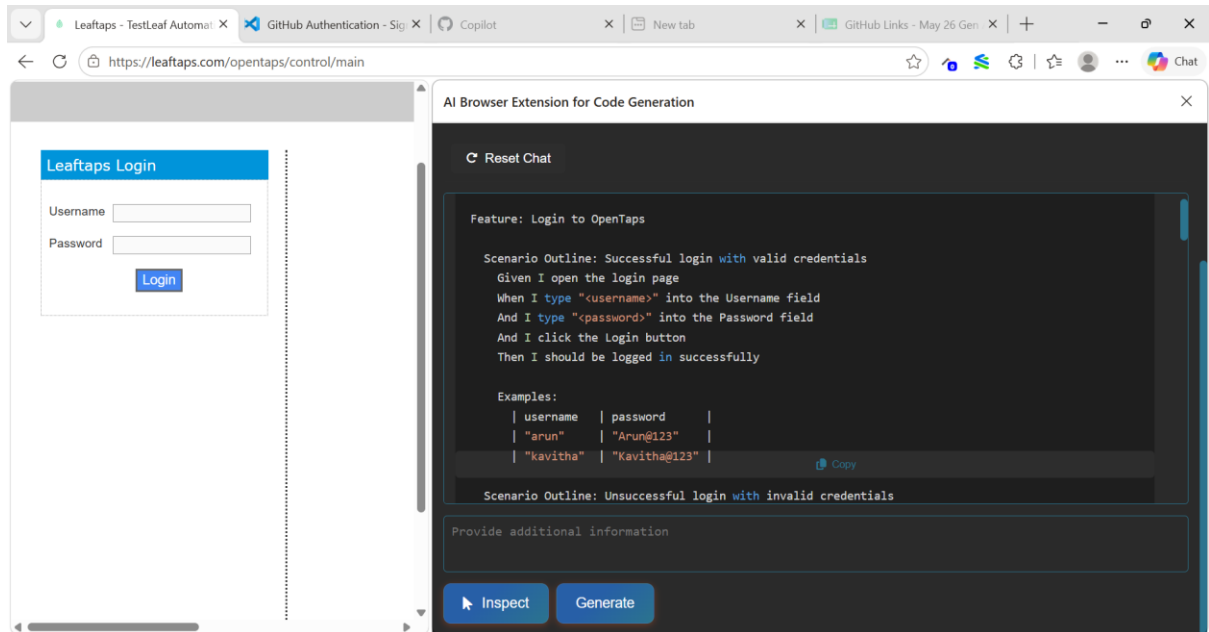
Inspect Generate

Playwright/Typescript:





Feature File:



Page Class:

The screenshot shows a web browser with the URL <https://leafTaps.com/opentaps/control/main>. The page displays a "LeafTaps Login" form with fields for "Username" and "Password", and a "Login" button. An "AI Browser Extension for Code Generation" window is open on the right, showing a code editor with the following code:

```
import { Page, Locator } from '@playwright/test';

/**
 * Page Object Model for the login form on the OpenTaps application.
 *
 * This class encapsulates all interactions with the login form,
 * providing a clean API for automation engineers.
 */
export class LoginPage {
  readonly page: Page;
  readonly form: Locator;
  readonly usernameInput: Locator;
  readonly passwordInput: Locator;
  readonly submitButton: Locator;
}
```

Below the code editor, there is a text input field labeled "Provide additional information" and two buttons: "Inspect" and "Generate".

The screenshot shows the same web browser and "LeafTaps Login" form as the previous image. The "AI Browser Extension for Code Generation" window is open on the right, showing a code editor with the following code:

```
/**
 * Fills the username field.
 * @param username The username to enter.
 */
async fillUsername(username: string): Promise<void> {
  await this.usernameInput.fill(username);
}

/**
 * Fills the password field.
 * @param password The password to enter.
 */
async fillPassword(password: string): Promise<void> {
  await this.passwordInput.fill(password);
}

/**
```

Below the code editor, there is a text input field labeled "Provide additional information" and two buttons: "Inspect" and "Generate".

LeafTaps - TestLeaf Automai XGitHub Authentication - Sig XCopilotNew tabGitHub Links - May 26 Gen X

https://leafTaps.com/opentaps/control/main

Chat

LeafTaps Login

Username

Password

Login

AI Browser Extension for Code Generation

Reset Chat

```
/**
 * Retrieves the action attribute of the login form.
 * @returns The form action URL as a string.
 */
async getFormAction(): Promise<string> {
  return (await this.form.getAttribute('action')) ?? '';
}

/**
 * Clears the username and password fields.
 */
async clearFields(): Promise<void> {
  await this.usernameInput.fill('');
  await this.passwordInput.fill('');
}
```

Provide additional information

InspectGenerate